

UNIVERSITY OF THE ARMED FORCES - ESPE

Computer Science Department

Object Oriented Programming

36863

Homework05 - UMLReading

Slides 1.01 01

Slides 1.05 07

Slides 1.05 08

Slides 1.05 09

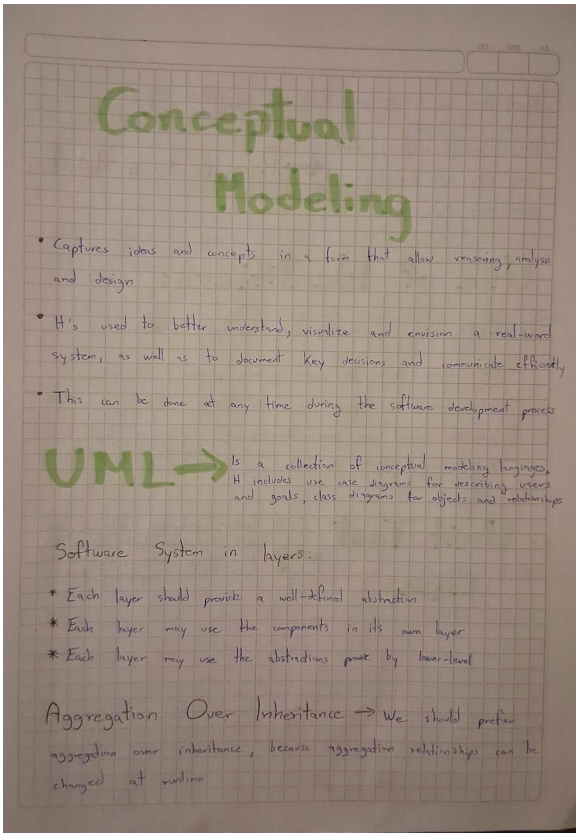
Slides 1.05 10

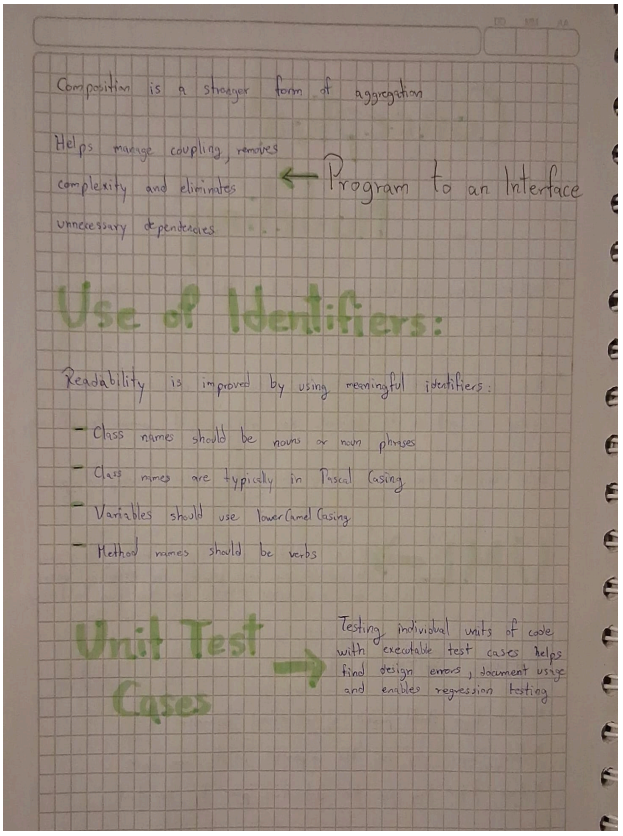
LASCANO, JORGE EDISON

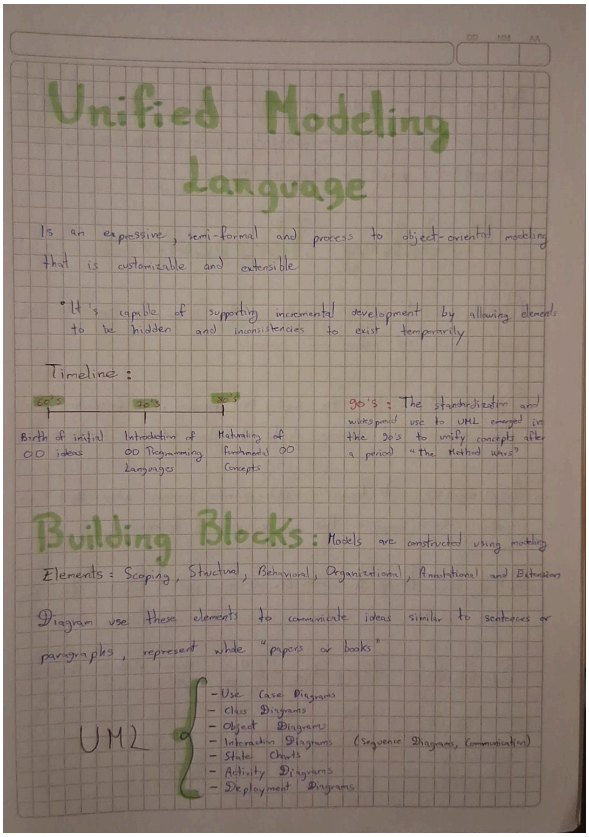
October 14, 2026

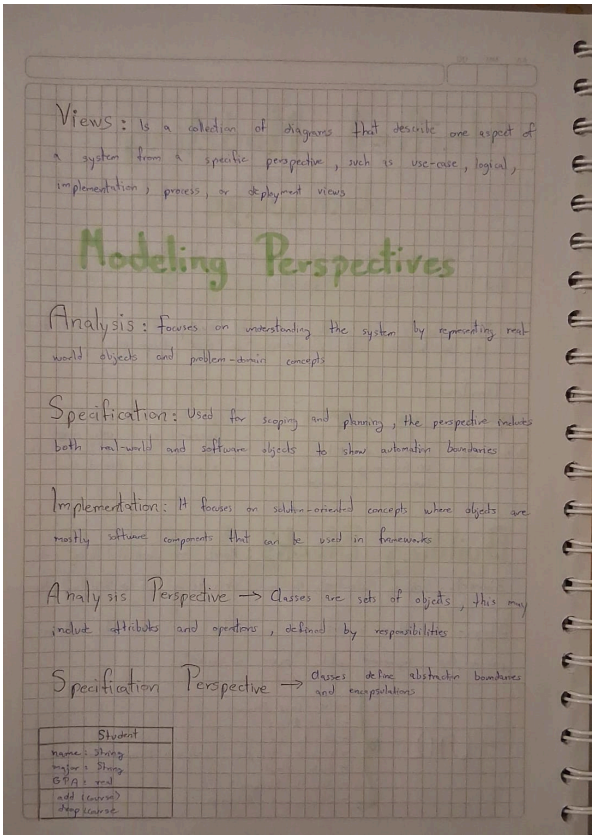
Slides Summary

Student: Jesus Garcia









Software Development Activities:

We accomplish the core software development activities

- Managing People & resources
- Conceptual Modeling
- Coding
- Testing
- Documenting
- Managing work artifacts

Types of Diagrams

Structual Diagrams — Used to illustrate static design or static process views of a system.

- Class Diagrams → Can be used to illustrate a static process view
- Object Diagrams → Address the static design view of static process view of a system
- Component Diagrams → Used to illustrate static implementation views of a system
- Deployment Diagrams → Used to illustrate static deployment views of an architecture

Behavioral Diagrams — These help organize and model the dynamic behavior of a system. They include use case diagrams, collaboration diagrams and more.

- Use Case Diagrams → Help organize and model the behavior of a system
- Sequence Diagrams → Show object and the messages that they send or receive
- Collaboration Diagrams → Are interaction diagrams that emphasize the structural
- State chart Diagrams → Used to model behavior of objects in terms of states
- Activity Diagrams → Specification and design views of a system

More on Views → Are collections of a diagrams that describe a system from a certain perspective
Use case views, logical views, implementation view, process view, deployment view

Modeling Different Levels of abstraction:

Different development activities require different levels of abstraction

Two ways:

- Using diagrams with different levels of detail
- Creating models of different levels of abstraction, but can be traced from one to another

Hints and Tips

- The true purpose of diagrams is to visualize, document and most importantly communicate, they aren't not just meant to be "pretty pictures"
- You should maintain a balance between structual and behavioral diagrams, give each diagram a clear and meaningful name, and avoid redundant diagrams

Use Cases

Actors: Represent external entities, such as users, other software, or hardware devices, that interact with the system to achieve specific goals

Use Cases: Captures a discrete user goal and documents the user-visible functionality of the system, independent of specific software. While system interactions describe how a system fulfills a goal, use cases focus strictly on the "what" and "Why" from the user's perspective

Diagram Components and Relationships:

Use case diagrams visually document these user goals using four primary components: actors, use cases, relationships and subjects

- Use cases relate to one another in three main ways: «extends», «includes» and generalization where one use case is a refinement of a broader one

Practical Hints and Benefits

- You can identify actors by asking who or what provides input, consumes output, or controls hardware connected to the system
- To find use cases, it is helpful to list the actors or external events first, and then determine the specific goals or expected system responses for each

Objects and Classes

Objects: Are the fundamental building blocks of object-oriented systems, representing abstractions that encapsulate state, behavior and identity

We use classes as abstraction to group together whole collections of objects that share common observable properties

These components can be modeled from three different perspectives: analysis, specification and implementation

Class Anatomy → Visually a class is represented as a box divided into compartments for it's name, attributes, operations and responsibilities

- Attributes are placeholders or slots that hold values to represent the characteristics of an object
- Operations define the behaviors of the class, representing how objects interact, the services they provide or their specific methods

Connecting System

Associations → These are structual relationships showing that objects of one class are connected to objects of another class. They use multiplicity constraints to dictate how many objects can be linked

Aggregation → Aggregation is a specific association meaning one class "is part of" another

Generalization → This represents a relationship where a specialized class is a subset or subtype of a broader, more general class

Dependencies → This relationship indicates that one component "uses" another, if the used component changes, the dependent one may also need to change

UML General Best Practice

- Maintaining an "Analysis" perspective by modeling the real-world, problem as stakeholders see it, rather than modeling software components too early

- You should let the model drive your exploration process, as you discover components, capture them, and let the incompleteness of the diagram trigger further questions

Diagram Types and Components

Use Case Diagrams: These model actors and their goals with respect to the system. They utilize <<include>> dependencies for required sub-goals and <<extend>> dependencies to model variations

Class Diagrams: Using initial and final nodes, action nodes, control/object flows, decision nodes, and fork/join nodes to handle concurrent threads

Interaction Diagrams: Communication and sequence diagrams, these model interactions from an object perspective, showing objects and the sequence

State Charts: These detail how an object's state changes over time using initial/final states, transitions and specific internal activities